DS605: Fundamentals of Machine Learning

Lab Assignment - 8

Decision Trees and Support Vector Machines

Dataset: UCI Banknote Authentication (1,372 samples, 4 features, 2 classes)

Submission: Public GitHub repository link only.

Objective

Implement and visualize Decision Tree and Linear SVM classifiers from scratch, compare them with Scikit-learn, and explore nonlinear SVM kernels.

Part A - Decision Tree from Scratch

- Using NumPy/Pandas, implement Gini impurity and entropy, choose feature-threshold splits, construct the tree recursively with stopping rules, and write a prediction function.
- Using variance and skewness, plot decision boundaries for the manual and Scikit-learn trees; also visualize the learned manual tree and its split rules.
- Explore criterion and maximum-depth settings; compare predictions and performance against Scikit-learn DecisionTreeClassifier.

Part B - Linear SVM from Scratch

- Scale input features using training-set statistics and map the binary labels to -1 and +1. Implement a linear soft-margin SVM using hinge loss and gradient/subgradient descent in NumPy.
- With the same two features, plot manual and Scikit-learn SVM decision boundaries. Show the separating line, +1/-1 margins, and margin-violating samples for the manual model.
- Compare against Scikit-learn linear SVM; study the effect of regularization (C), learning rate, and iterations.

Part C - Kernel SVM and Nonlinear Boundaries

- Generate a two-dimensional nonlinear dataset using make_moons or make_circles. Train Scikit-learn SVMs with linear, polynomial, and RBF kernels.
- Plot and compare all three decision boundaries, and explain why a nonlinear kernel may help.

Part D - Final Comparison

- On the original Banknote dataset, train manual and Scikit-learn Decision Trees, manual and Scikit-learn linear SVMs, and an RBF SVM using all four features.
- Use a consistent train-test split. Compare accuracy, precision, recall, F1-score, and training/prediction times; discuss interpretability, scaling, overfitting, and model complexity.

Expected Workflow

Banknote Data -> Two-Feature Models + Boundary Plots -> Manual Tree/SVM -> Kernel SVM -> Four-Feature Comparison -> Performance Analysis

Important Requirement

Use two features for 2D decision-boundary plots and all four features for final performance evaluation. Implement both from-scratch algorithms without Scikit-learn; plotting libraries and Scikit-learn comparisons are allowed.

Deliverables (GitHub Only)

- Public repository with runnable implementations, manual-tree visualization, all decision-boundary/margin plots, comparison results, and a concise README with observations.
- Submit only the public GitHub repository link.